

Furqan Memory Skill — Client-Side Storage & Reasoning Patterns

Architecture Document v1.0

Project: AI-Furqan Memory Skill — Local Knowledge Memory for AI Agents **Version:** 1.0 — Draft
Date: March 21, 2026 **Authors:** R&D/Innovation Lab **Status:** Proposed **Companion to:** Furqan Reasoning-as-a-Skill (RaaS) v1.0

1. Vision

1.1 The Problem

JI Reasoning Skill (RaaS) server-side — AI agent ويرد السيرفر بيفكر ويرد server-side — JI Reasoning Skill (RaaS) server-side

لكن فيه مشاكل:

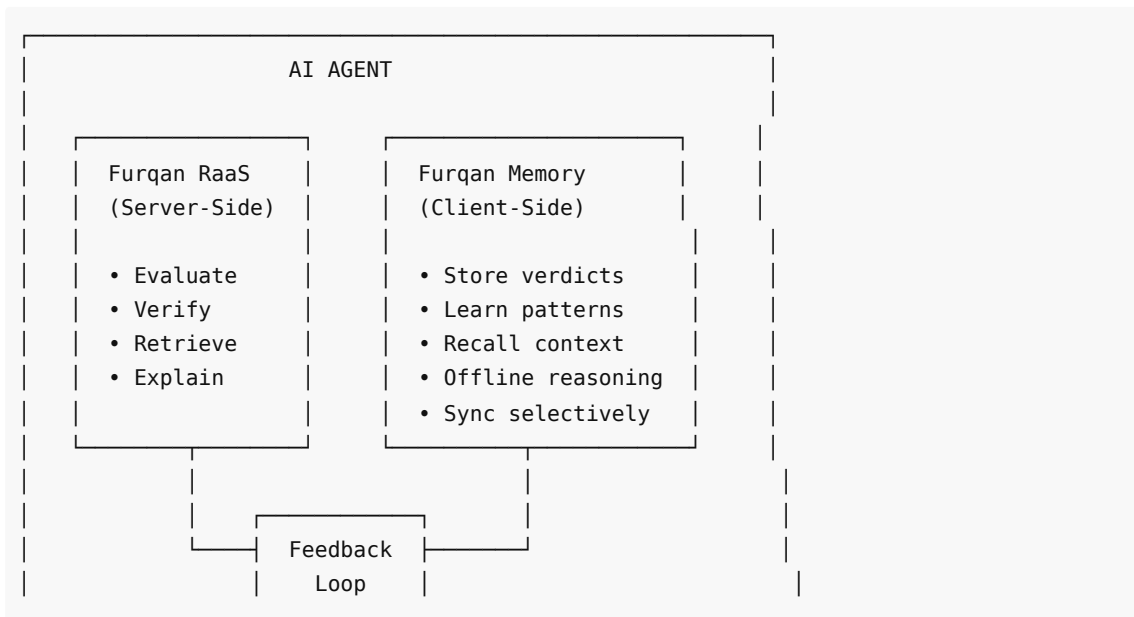
1. **Latency** — للسيرفر round-trip محتاجة query كل
2. **Privacy** — بيانات المستخدم بتطلع برا
3. **Offline** — reasoning من غير نت مفيش
4. **Cost** — provider بفلوس على AI call كل
5. **No Learning** — session بينسى كل حاجة كل AI agent

1.2 The Solution

Furqan Memory Skill — skill تانية مكملة بتشتغل **client-side** بالكامل:

- على جهاز المستخدم reasoning patterns وال verdicts بتخزن ال
- بيتعلم من الأنماط اللي اتكررت قبل كده ال agent
- معروف pattern لما ال offline بتشتغل
- Zero data leakage** — local كل حاجة

1.3 The Two-Skill Model

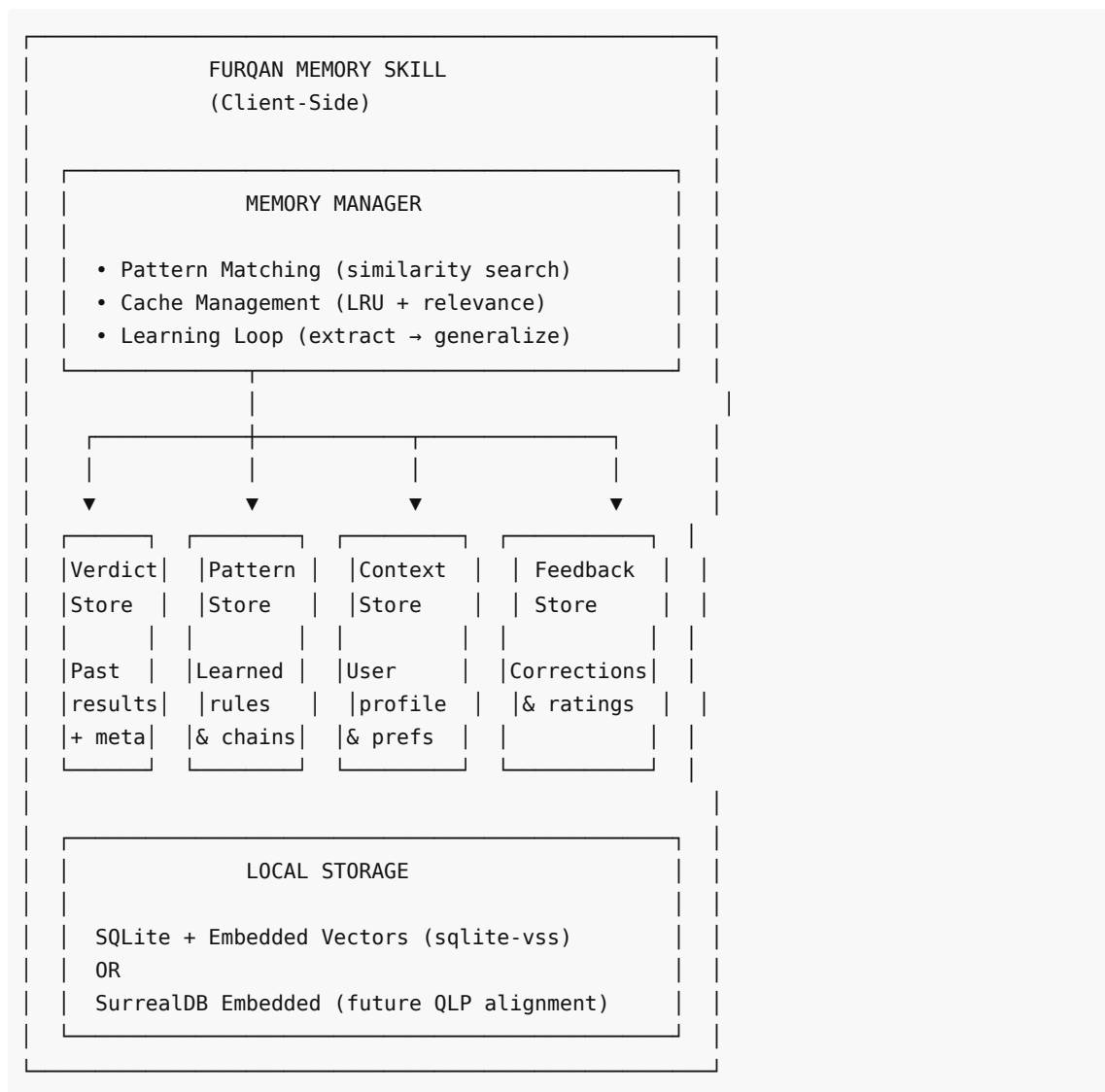


Flow:

1. Agent gets question
2. Memory Skill: "Have I seen this pattern before?" → YES → instant answer
3. Memory Skill: "No match" → RaaS Skill: full evaluation
4. Memory Skill: stores verdict + extracts pattern for next time

2. Architecture

2.1 Core Components



2.2 The Four Stores

Store	What It Holds	Purpose
-------	---------------	---------

Verdict Store	Past evaluation results with full metadata	Cache — avoid re-evaluating known questions
Pattern Store	Generalized reasoning patterns extracted from verdicts	Learning — recognize similar questions without exact match
Context Store	User preferences, domain focus, interaction history	Personalization — adapt responses to user's context
Feedback Store	User corrections, ratings, approvals/rejections	Refinement — improve pattern accuracy over time

3. Pattern System

3.1 What is a Reasoning Pattern?

A pattern is a **generalized rule** extracted from one or more verdicts.

Verdict (specific):

Q: "Is fractional reserve banking ethical?"

- Gate 1 FAIL (source: human theory, not divine)
- Gate 2 FAIL (debt multiplication contradiction)
- Gate 3 FAIL (founded on human preference)
- Score: 15/100

Pattern (generalized):

Category: Financial Systems

Rule: "Systems based on debt multiplication from fractional reserves"

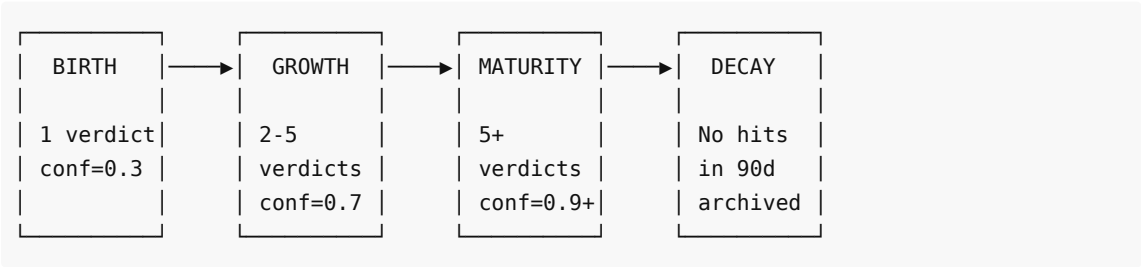
Signal: [human_theory_source, debt_multiplication, interest_based]

Expected Gates: [G1:FAIL, G2:FAIL, G3:FAIL]

Confidence: 0.92 (based on 3 similar verdicts)

Template: "Financial instruments involving {mechanism} that rely on interest-based debt creation fail Source Integrity (no divine endorsement) and Structural Consistency (mathematical contradiction in infinite growth assumption)."

3.2 Pattern Lifecycle



3.3 Pattern Data Model

```
@dataclass
class ReasoningPattern:
```

```

"""A generalized reasoning pattern learned from verdicts."""
id: str # UUID
category: str # Domain category
rule: str # Human-readable rule description
signals: list[str] # Keywords/concepts that trigger this pattern
expected_gates: dict[str, str] # Gate name → expected result
expected_score_range: tuple[int, int] # Min-max expected score
template: str # Response template with {placeholders}
confidence: float # 0.0 - 1.0
source_verdicts: list[str] # Verdict IDs this was learned from
hit_count: int # Times this pattern was matched
last_hit: float # Timestamp of last match
created_at: float # Birth timestamp
domain: str # Knowledge domain
feedback_score: float # Average user feedback rating
version: int # Pattern version (increments on update)

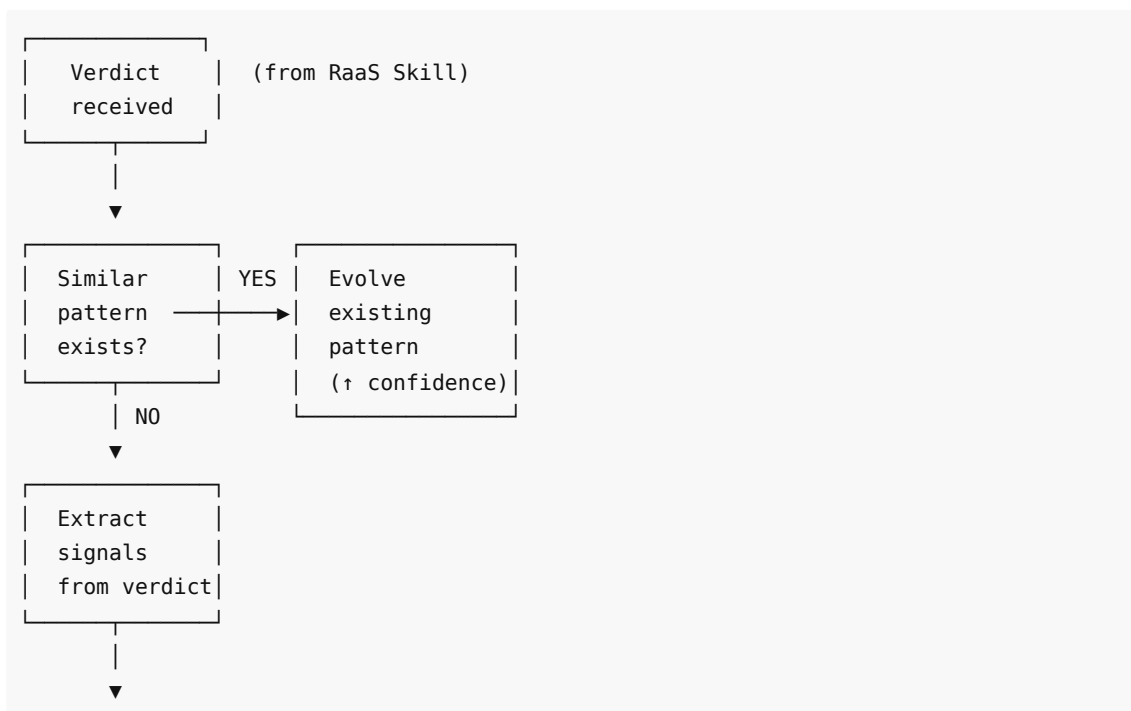
def matches(self, query: str, threshold: float = 0.75) -> float:
    """Return similarity score (0-1) for a query against this pattern."""
    ...

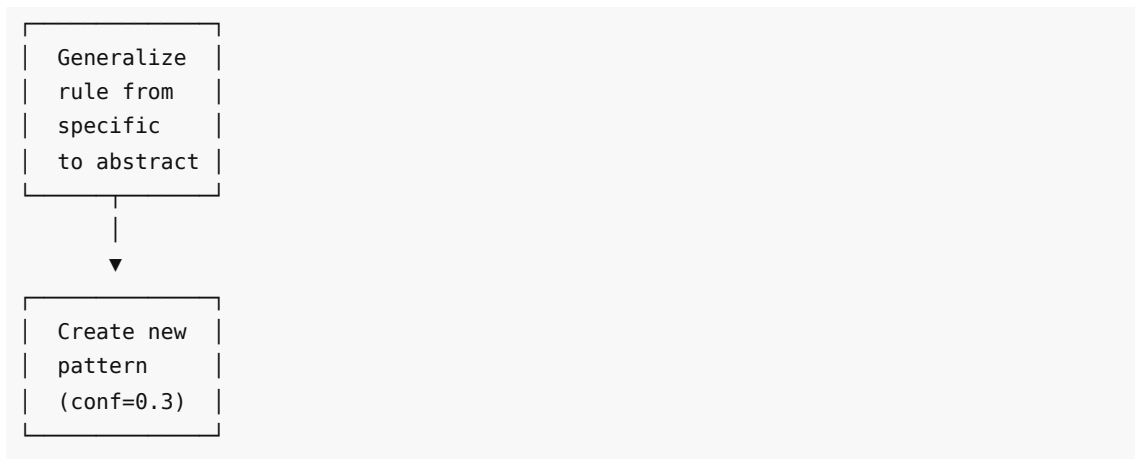
def apply(self, query: str) -> PreliminaryVerdict:
    """Apply this pattern to generate a preliminary verdict."""
    ...

def evolve(self, new_verdict: Verdict) -> None:
    """Update pattern confidence and template based on new verdict."""
    ...

```

3.4 Pattern Extraction Pipeline





4. Memory Operations (Skill Commands)

4.1 SKILL.md Definition

```
name: furqan-memory
description: >
  Client-side memory and learning skill for AI agents.
  Stores reasoning verdicts, learns patterns, enables
  offline reasoning, and personalizes responses.
  Works alongside furqan-reasoning (RaaS) skill.
  All data stays on the user's device.

commands:
  remember:
    description: Store a verdict or piece of knowledge in local memory
    params:
      verdict: object (required) – Full verdict from RaaS evaluation
      domain: string (default: "islamic")
      tags: list[string] – Additional categorization tags

  recall:
    description: Search memory for relevant past verdicts or patterns
    params:
      query: string (required)
      domain: string (default: "all")
      limit: int (default: 5)
      min_confidence: float (default: 0.5)

  recognize:
    description: Check if a query matches a known pattern (fast path)
    params:
      query: string (required)
      domain: string (default: "all")
      threshold: float (default: 0.75)

  reflect:
```

```

description: Trigger pattern extraction and learning from recent verdicts
params:
  since: string – ISO timestamp (default: last 24h)
  domain: string (default: "all")

forget:
description: Remove specific verdicts or patterns from memory
params:
  id: string – Verdict or pattern ID
  reason: string – Why it's being removed

feedback:
description: Rate a verdict or pattern (improves future accuracy)
params:
  id: string (required) – Verdict or pattern ID
  rating: enum [correct, partially_correct, incorrect]
  correction: string – What should have been different

stats:
description: Memory usage statistics
params:
  domain: string (default: "all")

export:
description: Export memory data (for backup or migration)
params:
  format: enum [json, sqlite]
  domain: string (default: "all")
  include_patterns: bool (default: true)

sync:
description: Selective sync with server (opt-in, privacy-first)
params:
  direction: enum [push, pull, both]
  domain: string (required)
  anonymize: bool (default: true) – Strip PII before push

```

4.2 MCP Server Tools

```

{
  "tools": [
    {
      "name": "furqan_remember",
      "description": "Store a reasoning verdict in local memory. Automatically
extracts patterns for future recognition.",
      "inputSchema": {
        "type": "object",
        "properties": {
          "verdict": { "type": "object", "description": "Full verdict object from
furqan_evaluate" },
          "domain": { "type": "string", "default": "all" },

```

```

    "tags": { "type": "array", "items": { "type": "string" } }
  },
  "required": ["verdict"]
}
},
{
  "name": "furqan_recall",
  "description": "Search local memory for relevant past verdicts and learned
patterns. Returns matches ranked by relevance and confidence.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "domain": { "type": "string", "default": "all" },
      "limit": { "type": "integer", "default": 5 },
      "min_confidence": { "type": "number", "default": 0.5 }
    },
    "required": ["query"]
  }
},
{
  "name": "furqan_recognize",
  "description": "Fast-path: check if a query matches a known reasoning pattern.
If matched, returns preliminary verdict without server call. Saves cost and
latency.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "domain": { "type": "string", "default": "all" },
      "threshold": { "type": "number", "default": 0.75 }
    },
    "required": ["query"]
  }
},
{
  "name": "furqan_reflect",
  "description": "Trigger learning: analyze recent verdicts and extract/update
reasoning patterns.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "since": { "type": "string", "description": "ISO timestamp" },
      "domain": { "type": "string", "default": "all" }
    }
  }
},
{
  "name": "furqan_feedback",
  "description": "Rate a past verdict or pattern to improve future accuracy.",
  "inputSchema": {
    "type": "object",

```

```

    "properties": {
      "id": { "type": "string" },
      "rating": { "type": "string", "enum": ["correct", "partially_correct",
"incorrect"] },
      "correction": { "type": "string" }
    },
    "required": ["id", "rating"]
  }
},
{
  "name": "furqan_memory_stats",
  "description": "Get memory usage statistics: verdict count, pattern count,
storage size, domain breakdown.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "domain": { "type": "string", "default": "all" }
    }
  }
}
}
]
}

```

5. Agent Workflow Integration

5.1 Full Flow: Memory + RaaS Together

User asks: "Is dropshipping halal?"

STEP 1 – RECOGNIZE (Memory Skill, local, <50ms)

```
↳ furqan recognize(query="dropshipping halal")
```

↳ Check patterns → Match found?

```

| YES (confidence ≥ 0.85)
|
| Pattern: "E-commerce models without inventory"
| Confidence: 0.88
| Expected: G1:Survive, G2:Conditional, G3:Survive
| Template: "Trading models where the seller..."
|
| → Agent uses pattern as basis
| → Adds disclaimer: "Based on similar past analysis"
| → Asks: "Want a fresh full evaluation?"

```

```

| NO (no match or confidence < 0.85) |
| STEP 2 - RECALL (Memory Skill, local, <200ms) |
|   ↳ furqan recall(query="dropshipping halal") |

```



```

└─ Any similar verdicts in history?
   → Returns relevant context for RaaS

STEP 3 – EVALUATE (RaaS Skill, server, ~5s)
└─ furqan_evaluate(question="dropshipping halal")
└─ Full 4-gate evaluation with sources

STEP 4 – REMEMBER (Memory Skill, local, <100ms)
└─ furqan_remember(verdict=<result>)
└─ Stores verdict + extracts/updates patterns

```

5.2 Offline Mode

```

OFFLINE MODE

RaaS Server unavailable?

1. furqan_recognize → pattern match?
   YES → Use pattern (with offline disclaimer)
   NO  → Step 2

2. furqan_recall → similar verdict?
   YES → Adapt past verdict (with disclaimer)
   NO  → "I need server access for a fresh
         evaluation. Here's what I know from
         similar topics: ..."

Never fabricate a verdict in offline mode.
Honesty > helpfulness.

```

5.3 Learning Over Time

```

Week 1:  Agent evaluates 50 questions via RaaS
         Memory stores 50 verdicts
         Extracts 12 patterns (confidence: 0.3-0.5)

Week 4:  200 verdicts stored
         30 patterns, 8 mature (confidence: 0.85+)
         ~40% of queries answered from patterns (fast path)

Month 3: 500 verdicts
         50 patterns, 20 mature
         ~60% fast-path answers
         Server cost reduced 60%

```

Month 6: Agent has domain expertise comparable to a junior researcher in the configured domains

6. Data Storage

6.1 Storage Engine Options

Option	Pros	Cons	Best For
SQLite + sqlite-vss	Zero dependency, single file, battle-tested	Vector search is basic	MVP, embedded, mobile
SurrealDB Embedded	Graph + vector + relational in one, QLP-aligned	Newer, less battle-tested	QLP v3.0 target
DuckDB	Fast analytics, columnar	No vector search native	Analysis workloads

Recommended path: SQLite + sqlite-vss for MVP → SurrealDB Embedded for QLP alignment.

6.2 Schema (SQLite)

```
-- Verdicts table
CREATE TABLE verdicts (
  id TEXT PRIMARY KEY,
  question TEXT NOT NULL,
  domain TEXT NOT NULL DEFAULT 'islamic',
  verdict_json TEXT NOT NULL,      -- Full verdict as JSON
  total_score INTEGER,
  gate_results TEXT,              -- JSON: {"G1":"Fail","G2":"Survive",...}
  final_judgment TEXT,
  tags TEXT,                      -- JSON array
  created_at REAL NOT NULL,
  accessed_at REAL,
  access_count INTEGER DEFAULT 0
);

-- Patterns table
CREATE TABLE patterns (
  id TEXT PRIMARY KEY,
  category TEXT NOT NULL,
  domain TEXT NOT NULL DEFAULT 'islamic',
  rule TEXT NOT NULL,
  signals TEXT NOT NULL,          -- JSON array
  expected_gates TEXT NOT NULL,   -- JSON object
  expected_score_min INTEGER,
  expected_score_max INTEGER,
  template TEXT,
  confidence REAL DEFAULT 0.3,
  source_verdicts TEXT,           -- JSON array of verdict IDs
  hit_count INTEGER DEFAULT 0,
```

```

    last_hit REAL,
    created_at REAL NOT NULL,
    feedback_score REAL DEFAULT 0.0,
    version INTEGER DEFAULT 1
);

-- Feedback table
CREATE TABLE feedback (
    id TEXT PRIMARY KEY,
    target_id TEXT NOT NULL,          -- Verdict or pattern ID
    target_type TEXT NOT NULL,       -- 'verdict' or 'pattern'
    rating TEXT NOT NULL,            -- 'correct', 'partially_correct', 'incorrect'
    correction TEXT,
    created_at REAL NOT NULL
);

-- Context table (user preferences)
CREATE TABLE context (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL,             -- JSON
    domain TEXT DEFAULT 'global',
    updated_at REAL NOT NULL
);

-- Vector embeddings (sqlite-vss)
CREATE VIRTUAL TABLE verdict_vectors USING vss0(
    embedding(384)                  -- MiniLM or CamelBERT dimension
);

CREATE VIRTUAL TABLE pattern_vectors USING vss0(
    embedding(384)
);

```

6.3 Storage Limits & Cleanup

```

STORAGE_CONFIG = {
    "max_verdicts": 10_000,          # Per domain
    "max_patterns": 1_000,           # Per domain
    "verdict_ttl_days": 365,         # Auto-archive after 1 year
    "pattern_decay_days": 90,        # Archive if no hits in 90 days
    "max_storage_mb": 500,           # Total storage limit
    "cleanup_strategy": "lru",       # Least recently used
    "backup_on_cleanup": True,       # Archive before delete
}

```

7. Privacy & Security

7.1 Core Principles

Principle	Implementation
Local-First	All data stored on user's device. Zero server requirement for memory operations
Zero PII in patterns	Pattern extraction strips personal identifiers before generalization
Opt-in sync only	Server sync is explicit, never automatic
Anonymization	Sync pushes anonymized patterns, not raw verdicts
Encryption at rest	SQLite database encrypted with user key (SQLCipher)
User ownership	Export/delete anytime. No vendor lock-in

7.2 Sync Protocol (Opt-in)

SYNC FLOW

User explicitly calls:
furqan_sync(direction="push", anonymize=true)

1. Select mature patterns (confidence > 0.8)

2. Strip all PII and user-specific context

3. Generalize: specific question → abstract

4. Push to server as "community patterns"

5. Server aggregates from multiple users

6. Pull: download high-confidence community patterns to enrich local memory

Result: Collective intelligence without exposing individual data.

7.3 What NEVER Leaves the Device

- Raw user questions
- Personal context (names, locations, specific situations)
- Verdict details with identifiable info
- Feedback text containing personal details
- Interaction timestamps and frequency patterns

7.4 What CAN Be Synced (opt-in + anonymized)

- Generalized patterns (abstract rules, no specifics)
- Gate outcome distributions (statistical, not individual)
- Domain coverage maps (what topics are well-covered)

8. Performance Targets

Operation	Target Latency	Notes
recognize (pattern match)	< 50ms	Local vector search + threshold check
recall (memory search)	< 200ms	Semantic search over local verdicts
remember (store verdict)	< 100ms	Write + async pattern extraction
reflect (learning)	< 2s	Batch pattern extraction, can be deferred
feedback (rate)	< 50ms	Simple write
stats	< 100ms	Aggregate query

8.1 Storage Footprint

Scale	Verdicts	Patterns	DB Size	RAM Usage
Light	100	20	~5 MB	~20 MB
Medium	1,000	100	~50 MB	~80 MB
Heavy	10,000	500	~200 MB	~150 MB
Max	50,000	1,000	~500 MB	~250 MB

9. Implementation Roadmap

Phase 1: Core Memory (Week 1-2)

- ☐ SQLite schema + migrations
- ☐ Verdict Store: CRUD operations
- ☐ Embedded vector search (sqlite-vss)
- ☐ remember and recall commands
- ☐ Basic SKILL.md for agent integration

Phase 2: Pattern Learning (Week 3-4)

- ☐ Pattern extraction from verdicts
- ☐ Signal detection (keyword + semantic)
- ☐ Pattern matching (recognize command)
- ☐ Confidence scoring + lifecycle management
- ☐ reflect command for batch learning

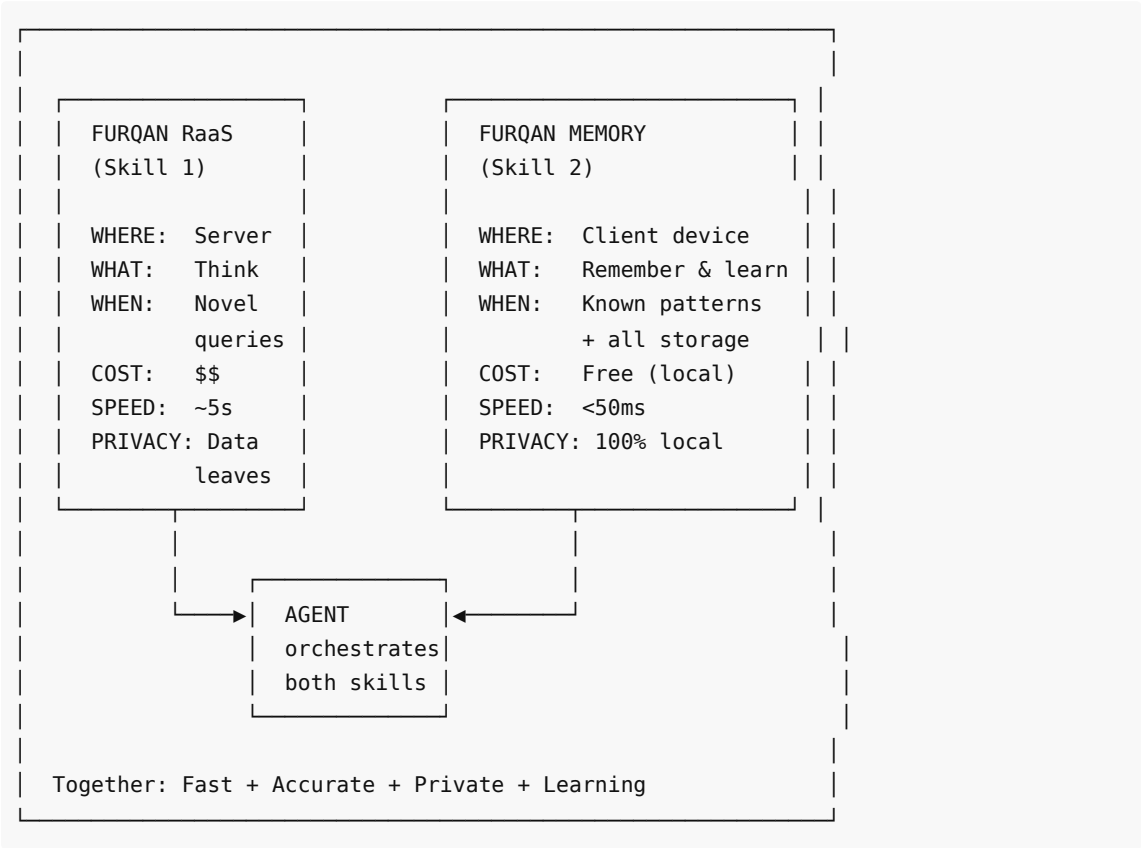
Phase 3: Feedback & Refinement (Week 5-6)

- ☐ Feedback collection + storage
- ☐ Pattern evolution based on feedback
- ☐ Confidence adjustment algorithms
- ☐ Pattern decay + cleanup
- ☐ feedback and stats commands

Phase 4: Sync & Production (Week 7-8)

- ☐ Anonymization pipeline
- ☐ Opt-in sync protocol
- ☐ Community pattern aggregation (server-side)
- ☐ SQLCipher encryption
- ☐ Export/import functionality
- ☐ MCP server packaging

10. Relationship to RaaS Skill



10.1 Commercial Model

Component	Pricing	Rationale
Memory Skill	Free / Open Source	Runs on user's device, encourages adoption
RaaS Skill	Paid per evaluation	Server resources, LLM costs, knowledge curation
Community Patterns	Free tier + Premium	Basic patterns free, specialized domains paid
Custom Adapters	Enterprise license	Domain-specific knowledge bases

Strategy: Memory Skill is the **trojan horse** — free, local, useful. It drives adoption of RaaS (paid) for novel queries the memory can't handle yet.

11. QLP v3.0 Alignment

QLP Principle	Memory Skill Implementation
Local-First	✔ All data on device, SQLite/SurrealDB embedded
User Sovereignty	✔ User owns all data, export anytime, no vendor lock
CRDTs	🌀 Future: CRDT-based sync for multi-device memory
Sovereign Stack	✔ No external dependency for core operations
Privacy by Design	✔ Zero PII leakage, encrypted at rest
SurrealDB	🌀 Migration path from SQLite → SurrealDB Embedded

12. Differentiation

Feature	Typical AI Memory	Furqan Memory Skill
Storage	Cloud (vendor-owned)	Local (user-owned)
Learning	None / basic RAG	Pattern extraction + generalization
Privacy	Data leaves device	100% local, opt-in sync only
Verification	None	Patterns carry gate scores + confidence
Offline	✗	✔ Full offline capability
Interop	Single vendor	MCP standard, any agent
Cost	Per-query forever	Decreasing (patterns reduce server calls)
Feedback loop	✗	✔ Human feedback improves patterns

This document is a living draft. Updated as architecture evolves. Companion document: FURQAN-REASONING-AS-A-SKILL-v1.0.md